



Running OpenAI's Agents API on MAG

A translation layer, tested end-to-end against the official SDK

Huihuo Zheng & **Trinity**

Argonne Leadership Computing Facility (ALCF)

September 14, 2026

*Trinity — AI Agent for Autonomous Scientific Discovery in HPC,
developed by Huihuo Zheng.*

huihuo.zheng@anl.gov

Agenda

- ❓ The two questions we set out to answer
- 🔧 What MAG actually serves (live tests)
- 🔗 Agents *SDK* vs. Agents *API* — the crux
- 🖼️ The translation layer, in one picture
- ✅ What's built and proven
- ⚙️ Inside the adapter: loop & key details
- 🧭 Two paths, one decision
- 📢 The one ask to OpenAI
- 🔑 Getting started: MAG token & run
- 🚩 Recommendation & the repo

Two questions drove all of this work

❓ Question 1 — feasibility

Can OpenAI's **agentic Codex / Agents API** run against the **AMSC Model Access Gateway (MAG)** — at the scale of a 50B-token run?

📢 Question 2 — the ask

If something is missing, **what precisely** do we ask OpenAI for — and what can we ship *today* without waiting on them?

We answered both by **testing MAG live** and **building the missing piece**.

MAG already serves every primitive the Agents API is built on

Capability	MAG endpoint	Live
Model inference (Responses wire API)	POST /v1/responses	✓ 200
OpenAI-compatible fallback	POST /v1/chat/completions	✓ 200
Model-emitted tool calls	output:[function_call]	✓
Stateful sessions	previous_response_id	✓
Streaming	stream:true → SSE	✓
Context compaction	POST /v1/responses/compact	✓
Managed Agents API	POST /v1/agents/sessions	✗ not impl.

i MAG is a **model gateway** (Kong → LiteLLM): it serves *inference*. The managed Agents API is OpenAI-hosted *orchestration* — a gateway cannot re-serve it.

The crux: two OpenAI “agents” products, different problems

Agents *SDK*

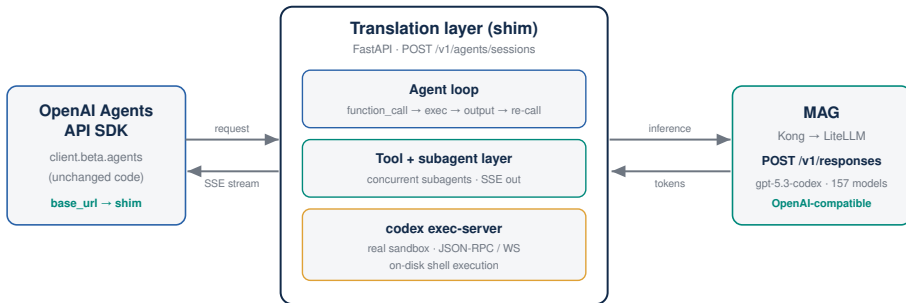
`openai-agents:` Agent, Runner, ModelProvider. Open-source, **client-side**. Loop runs in *your* process. Has a `base_url` **hook** — points at MAG directly, *today*, ~20 lines, no adapter.

Agents *API*

`client.beta.agents,` POST
`/v1/agents/sessions.` OpenAI's **managed cloud** service. Loop runs on *OpenAI's* servers. **No** `base_url` **hook** — cannot point at MAG. **This is the gap the adapter fills.**

Andrew Schmeder (LBNL) shared the SDK path; our work targets the *managed API* that has no hook.

The adapter re-serves the managed loop on top of MAG



The shim owns the agent loop + tools; MAG serves only inference. Only the client's **base_url** changes.

What's built and proven — the full chain executes live

✔ Core — tested and passing

- ✔ Agents API contract → loop over MAG /v1/responses
- ✔ `function_call` → `exec` → `output` → `re-call`
- ✔ **Real sandbox** (codex `exec-server`), on-disk effects
- ✔ **Concurrent subagents**, each its own sandbox
- ✔ Queue-based SSE, clean event interleave

✔ Shipped since — all live-tested

- ✔ Native streaming passthrough
- ✔ `/responses/compact` wiring
- ✔ `apply_patch` / `fs` tools
- ✔ Sandbox-policy hardening
- ✔ Exact event schema, auth, persist, cancel
- ✔ **Local (stdio) MCP servers via `exec-server`**

The proof: OpenAI's own SDK examples run *unchanged*

Example (Rick Stevens' repo)	Exercises	Result
01_basic_session.py	SDK non-stream session	✓ exit 0
03_multi_agent_review.py	streaming + multi-agent	✓ 2 subagents
05_raw_curl.sh	raw HTTP/SSE lifecycle	✓ full cycle

🔑 The **only** change is `OPENAI_BASE_URL=http://127.0.0.1:8811/v1`. The example code is byte-for-byte upstream. Session objects + streaming events validated against real `openai==3.13.0` models — **24/24**.

Inside the adapter: the server-side agent loop

The per-turn loop

1. Client POST `/v1/agents/sessions` → shim opens SSE, emits `turn.in_progress`
2. Shim calls MAG `/v1/responses (stream:true)`; forwards token deltas *live*
3. Model emits `function_call` → dispatch to sandbox tool → feed output back → re-call MAG
4. Repeat until a final message → `turn.completed`

Module map

<code>mag_agents_shim.py</code>	loop, SSE, sessions, cancel
<code>agents_api_models.py</code>	exact event/item builders
<code>codex_sandbox.py</code>	exec-server, fs, patch
<code>mcp_stdio.py</code>	MCP over exec-server

The loop OpenAI runs server-side — re-served on top of MAG inference.

Three details that made the translation faithful

🛡️ Real sandbox

Tools run in codex exec-server, launched **hardened** (disk-write-cwd + read-only elsewhere), each session its own workspace. A persistent-process **stdio bridge** (process/start+process/write) even drives local **MCP servers** in-sandbox.

✈️ Compaction quirk

MAG /responses/compact **rejects its own** >64-char response IDs, so it can't chain by previous_response_id. Worked around via the {model,input} form; the non-input compaction item is cleaned back into reusable history.

☰ Faithful wire

Streaming is a **live passthrough** of MAG deltas (not post-hoc chunking). A **cancel-race guard** aborts mid-tool, not just at turn boundaries. Every event/item validated against real openai==3.13.0 — **24/24**.

Each was reverse-engineered from MAG / the exec-server live, then locked down with a test.

Two paths — the decision hinges on where the 50B tokens live

OpenAI-platform grant

Managed Agents API works **natively** — but on **OpenAI's models**; MAG is not in the loop. (You may still self-host the *sandbox* near your data.)

AMSC / MAG allocation

Use the **self-hosted Codex harness** (or this adapter) pointed at MAG. The managed API cannot be used without the product change on the next slide.

For the 50B-token run *now*: self-host against MAG — **no OpenAI dependency**.

The one ask to OpenAI — and what already works today

 **Add a custom-model-provider / base_url to the Agents API**, so the managed harness can route inference to an OpenAI-compatible endpoint (MAG) instead of OpenAI's hosted models.

Need	Route
Agents on MAG, you control the client	Agents <i>SDK</i> + <code>ModelProvider(base_url)</code> — no infra
Existing <i>managed-API</i> code on MAG	this adapter , until OpenAI ships the hook

The ask is *only* for the managed Agents API. The SDK path needs no product change.

Before a 50B-token run: four items to close with AMSC

- 🕒 **Per-key concurrency + TPM/RPM** — agentic Codex is bursty; one agent opens dozens of concurrent requests.
- ↔ **Is /v1/responses rate-limited differently** from /v1/chat/completions?
- 📈 **Sustained throughput headroom** for 50B tokens (tokens/sec; a batch / priority lane?).
- 📊 **Server-side usage accounting** to track burn against the allocation.

MAG docs currently list no rate limits or quotas (“usage info coming soon”).

Recommendation

Ship now

Self-host the Codex harness (or this adapter) against MAG for the 50B-token run. **No OpenAI dependency.**

Press OpenAI

For managed-API portability, ask for the `base_url` hook — bring this repo as evidence the integration is straightforward.

 **Working proof:** `github.com/zhenghh04/amsc-mag-agents-api-adapter`

Public, secret-free; SDK examples reproduce with `bash scripts/run_examples.sh`.

Getting started: get a MAG token, then run

🔑 1 — Get a MAG token (PAT)

- Log in at `portal-lite.genesis.american-science-cloud.org` (Ping Identity)
- Select your **Genesis Mission RFA** project → **switch**
- **Personal Access Tokens** → **Generate PAT** → copy once

❗ Same PAT works for both wire APIs:
OpenAI: `.../v1` ▪ Anthropic: `.../` (root)

>_ 2 — Configure & run

```
cp env.example .env
# set MAG_API_KEY = your PAT
# sanity check:
curl .../v1/models \
-H "Authorization: Bearer $KEY"

bash scripts/run_all.sh
bash scripts/run_examples.sh
```

Docs: [amsc-docs-d762d2.gitlab.io/](https://amsc-docs-d762d2.gitlab.io/model-access-gateway)
model-access-gateway

Acknowledgement

This research used resources of the Argonne Leadership Computing Facility, a U.S. Department of Energy (DOE) Office of Science user facility at Argonne National Laboratory and is based on research supported by the U.S. DOE Office of Science-Advanced Scientific Computing Research Program, under Contract No. DE-AC02-06CH11357.

Thank you

Questions?

 github.com/zhenghh04/amsc-mag-agents-api-adapter ■  huihuo.zheng@anl.gov